

CXT418 Designing Human-Centered Systems

Module 5

Mental Models and Conceptual Design

What Is a Mental Model?

- “A mental model is an explanation of someone’s thought process about how something works in the real world.”
- It is an individual’s internal explanation of how a system works.
- A designer’s mental model is a vision of how a system works as held by the designer.
- A user’s mental model is a description of how the system works, as held by the user.
- It is the job of conceptual design to connect the two.

Designer's Mental Model

- Also called a **conceptual model**, it represents the designer's understanding of how a system is structured and functions.
- A well-defined designer's mental model is crucial for a clear and effective system design.
- A poorly formed model leads to confusion, vague user experiences, and usability issues.
- Designers use metaphors, analogies, and ontological structures to create a mental model that aligns with user expectations.

Three Perspectives of the Designer's Mental Model

1. Ecological Perspective:

- Describes **how the system fits into its broader environment** and workflow.
- Example: A **thermostat** is part of a larger heating system that includes a heat source, a heat distribution network, and a control unit.

2. Interaction Perspective:

- Describes **how users interact with the system** (physical actions, cognitive processes, feedback).
- Example: A thermostat has **buttons, dials, or touchscreens** to adjust the temperature, and it **provides feedback** on the display.

3. Emotional Perspective:

- Describes the **emotional impact** the design intends to create.
- Example: A sleek, intuitive **smart thermostat** (like the Nest) may evoke a **sense of modernity, convenience, and efficiency**.

User's Mental Model

A **user's mental model** is their **personal understanding** of how a system works, based on **past experiences, observations, and interactions**.

How Do Users Form Mental Models?

1. Knowledge in the Head:

- Comes from **past experiences, training, and expertise**.
- Example: A user who has used a traditional thermostat assumes **turning it up heats the room faster** (even if that's incorrect).

2. Knowledge in the World:

- Comes from **external sources** such as **interface design, cultural conventions, documentation, and social influences**.
- Example: A thermostat with a **display showing "Heating On"** provides real-time feedback, helping users form a **more accurate** mental model.

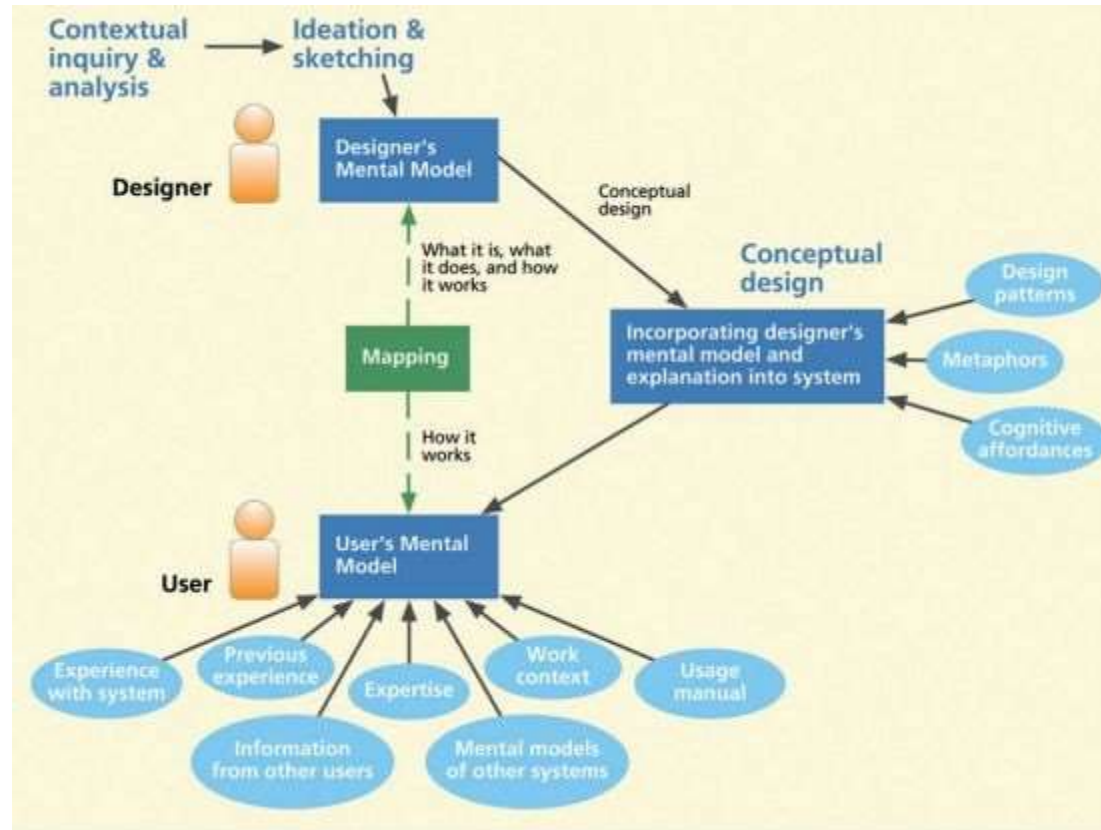
Challenges with User Mental Models

- **Users often apply past experiences** to new systems, which can lead to **misconceptions**.
- **If a system design does not clearly convey how it works**, users **guess** based on past knowledge, which may not always be correct.
- Example: In the 1992 movie *My Cousin Vinny*, Marisa Tomei's character struggles with a **rotary dial phone**, expecting it to work like a **touch-tone phone**—a classic case of **mental model mismatch**.

Mapping and the Role of Conceptual Design

- The goal of **conceptual design** is to align the designer's mental model with the user's mental model.
- Mismatched models lead to confusion and ineffective system interactions.
- Users may develop different and sometimes incorrect mental models of the same system.
- A well-designed system helps users form an accurate mental model through clear conceptual design.

Mapping the designer's mental model to the user's mental model.



CONCEPTUAL DESIGN

What Is a Conceptual Design?

A **conceptual design** is a high-level description of how a system functions, focusing on the **core idea or theme** behind the user experience.

- It **translates the designer's mental model** into a form users can understand.
- It serves as the "**system image**"—the only way designers and users communicate.
- A strong conceptual design ensures that the system is **intuitive, coherent, and easy to use**.

Why Is It Important?

- A poorly defined conceptual design leads to **confusion and usability problems**.
- Conceptual design lays the foundation for the **user experience (UX)**—it **cannot be fixed later** if it's wrong from the start.
- It helps establish a **theme, metaphor, or guiding principle** for the entire system.

Start with a Conceptual Design

Many designers **jump into screen layouts, menu structures, and UI elements** too early. Instead, they should start by defining **what the system is and how it should work**.

Best Practices for Starting Conceptual Design:

- Gather the **same team** that worked on **ideation and sketching**.
- Synthesize **all ideas** into a **high-level conceptual model**.
- Consider the **system's ecology, user interactions, and emotional impact**.

Different Starting Points for Conceptual Design:

- If the system operates in a **complex work environment**, start with the **ecological perspective**.
- If the system's **emotional appeal** is most important, start with the **emotional perspective**.
- If the system revolves around **a new interaction technique** (e.g., Apple's **iPod Classic scroll wheel**), start with the **interaction perspective**.

Leverage Metaphors in Conceptual Design

A **metaphor** is an analogy that helps users understand a new system by **relating it to something familiar**.

- Metaphors make interfaces **easier to learn** by leveraging **existing knowledge**.
- Instead of **reducing complexity**, they **help users manage complexity**.

Examples of Metaphors in Design

1. Metaphors in the Ecological Perspective (System as a Whole)

- **Example: iTunes as a "mother ship"** for iPods and iPhones.
 - iTunes manages all content (music, apps, videos) and **syncs** it across devices.
 - This metaphor helps users understand **iTunes as a central hub** for managing devices.

2. Metaphors in the Interaction Perspective (User Actions & Interface)

- **Example: A digital calendar app behaves like a paper calendar.**
 - Users can "write" events, move things around, and view schedules in a familiar way.
- **Example: iPad's page-turning animation mimics real book pages.**
 - This makes digital reading feel more natural.
- **Example: Apple's "Time Machine" backup feature.**
 - Users "travel back in time" visually to retrieve lost files.

3. Metaphors in the Emotional Perspective (Feelings & Branding)

- **Example: Garmin GPS as a "hiking companion."**
 - Marketing phrases like "Find yourself, then get back" evoke a sense of adventure and trust.
- **Example: A MINI Cooper's branding as a "fun and independent" car.**
 - MINI drivers associate their car with individuality and excitement.

Metaphor Breakdowns: When Metaphors Fail

- **Example: Dragging a disk to the Trash icon to eject it (Macintosh).**
 - Users expect the Trash to **delete files**, not eject disks.
 - This violates the **desktop metaphor**, causing confusion.

Conceptual Design from the Three Design Perspectives

1. Ecological Perspective: The System's Role in Its Environment

- Describes **how the system fits into its broader infrastructure**.
- Example: **Amazon Kindle**
 - The Kindle is **self-sufficient** and integrates seamlessly with Amazon's ecosystem.
 - It connects to the **Internet** to browse, buy, and download books.
 - It **syncs with other devices** to maintain a user's reading progress.

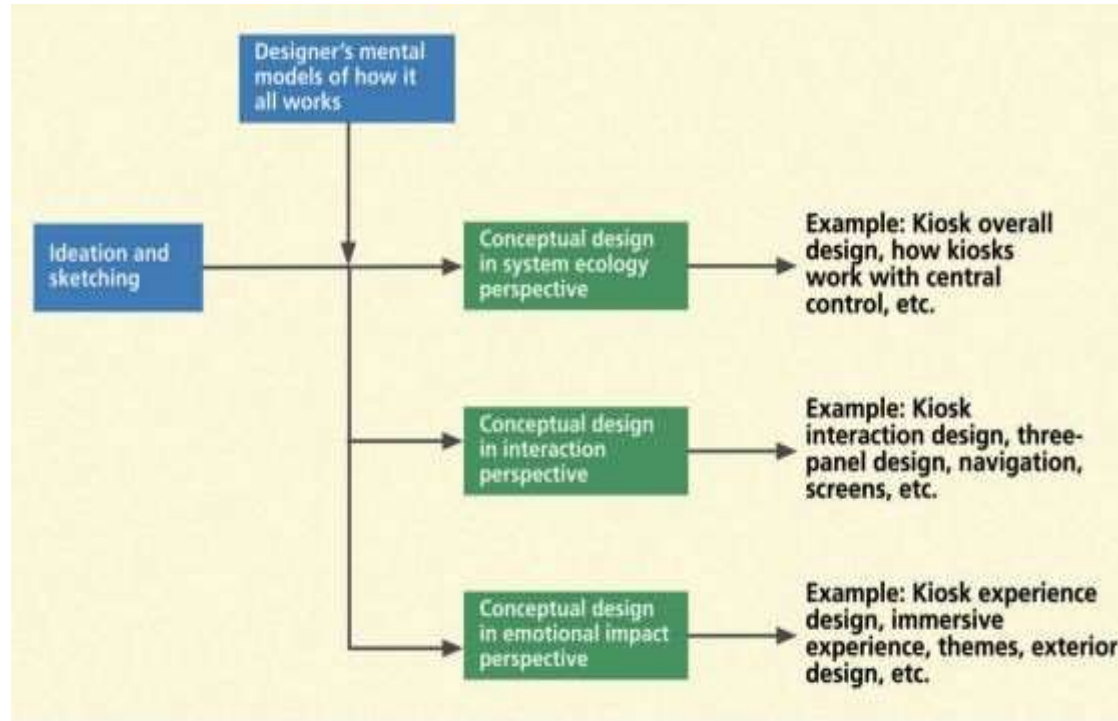
2. Interaction Perspective: How Users Operate the System

- Defines **how the user interacts with the system**.
- Example: **Apple's Time Machine (Mac Backup Feature)**
 - Users **"fly through time" visually** to recover lost files.
 - The interface is **intuitive and engaging**, making backup recovery effortless.
- Example: **Microsoft Outlook**
 - A familiar **three-pane layout** (navigation bar, list view, preview panel) makes it easy for users to adapt.

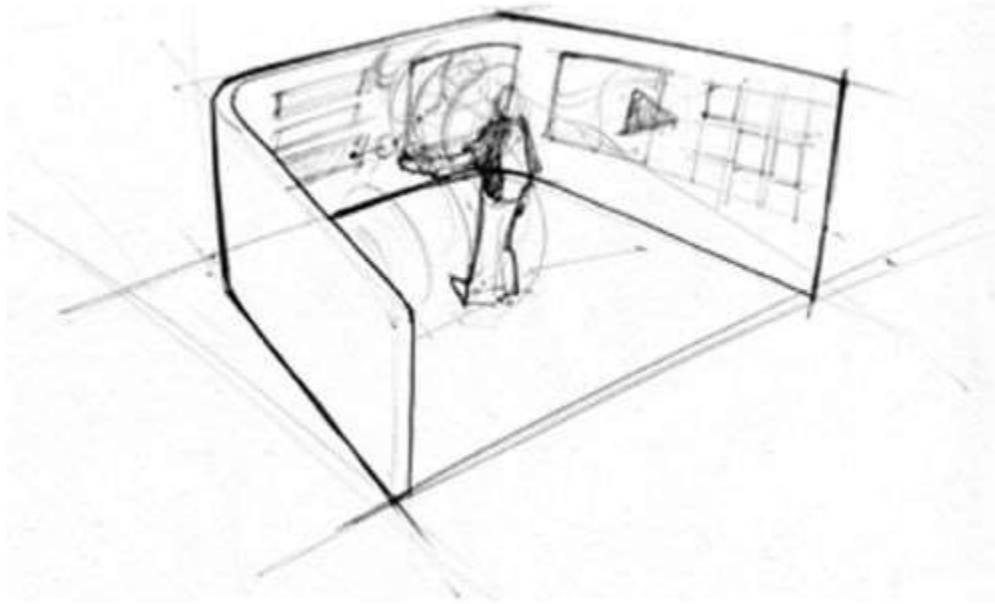
3. Emotional Perspective: How the System Feels to Users

- Defines the **emotional impact and experience**.
- Example: **Luxury Car Design**
 - A high-performance sports car aims to evoke **excitement and exclusivity**.
 - A MINI Cooper emphasizes **fun, uniqueness, and individuality**.

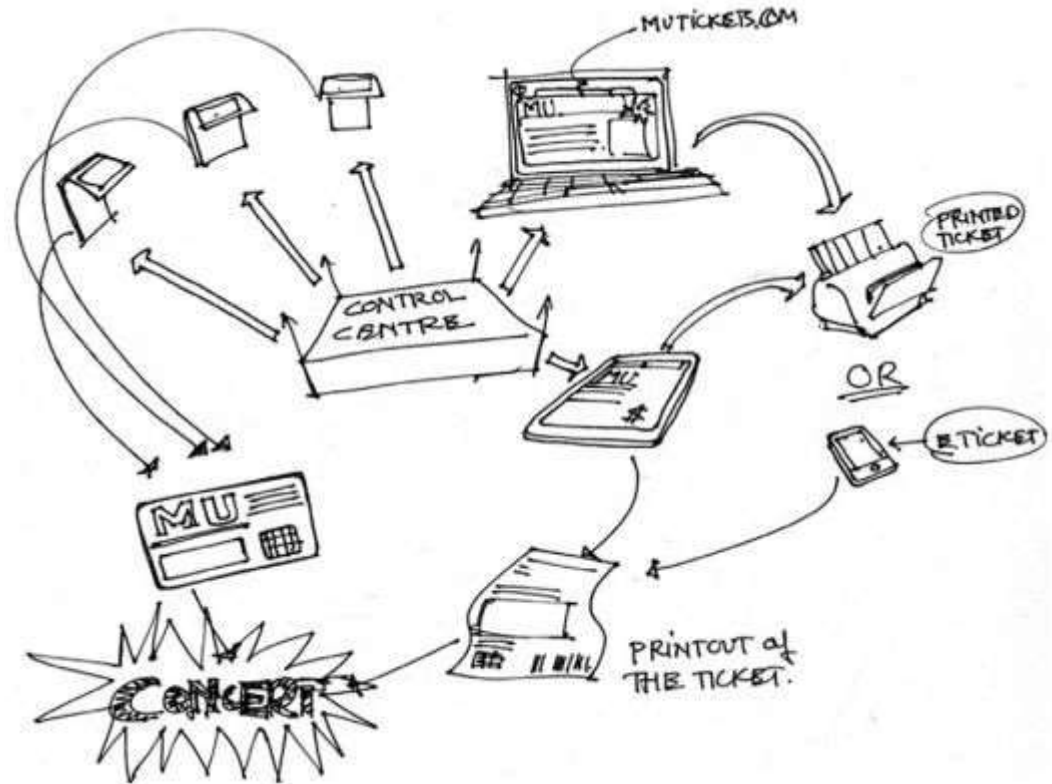
Designer workflow and connections among the three conceptual design perspectives.



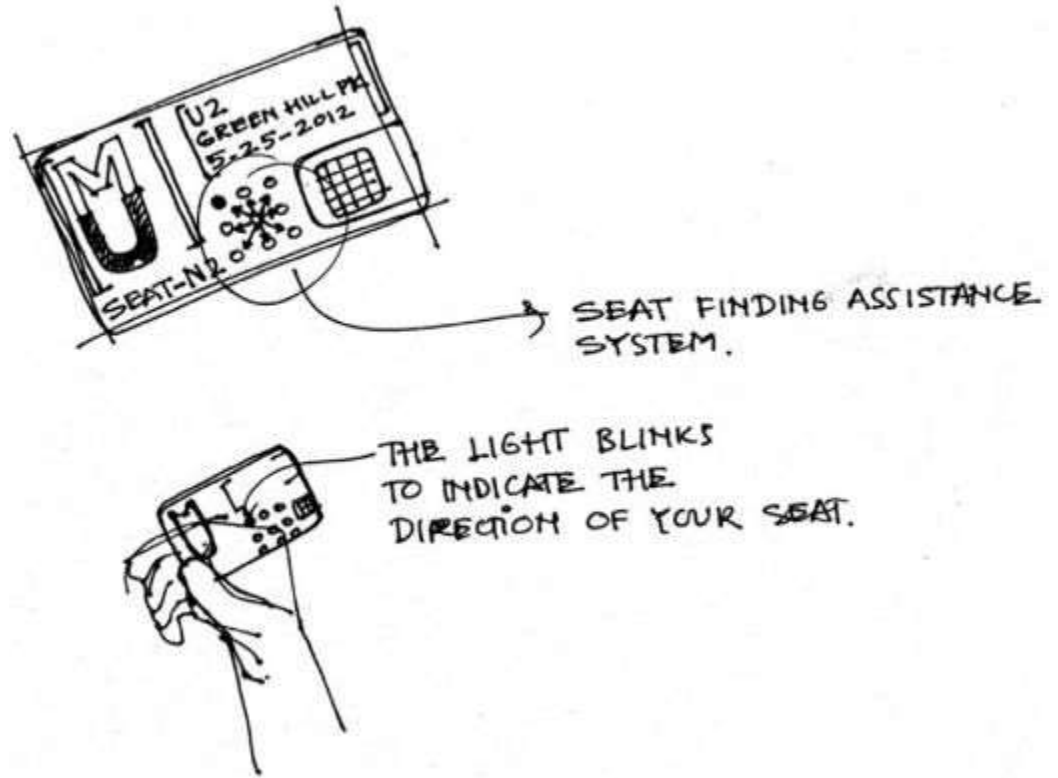
Part of a conceptual
design for the the
Ticket Kiosk System
showing
immersion in the
emotional perspective



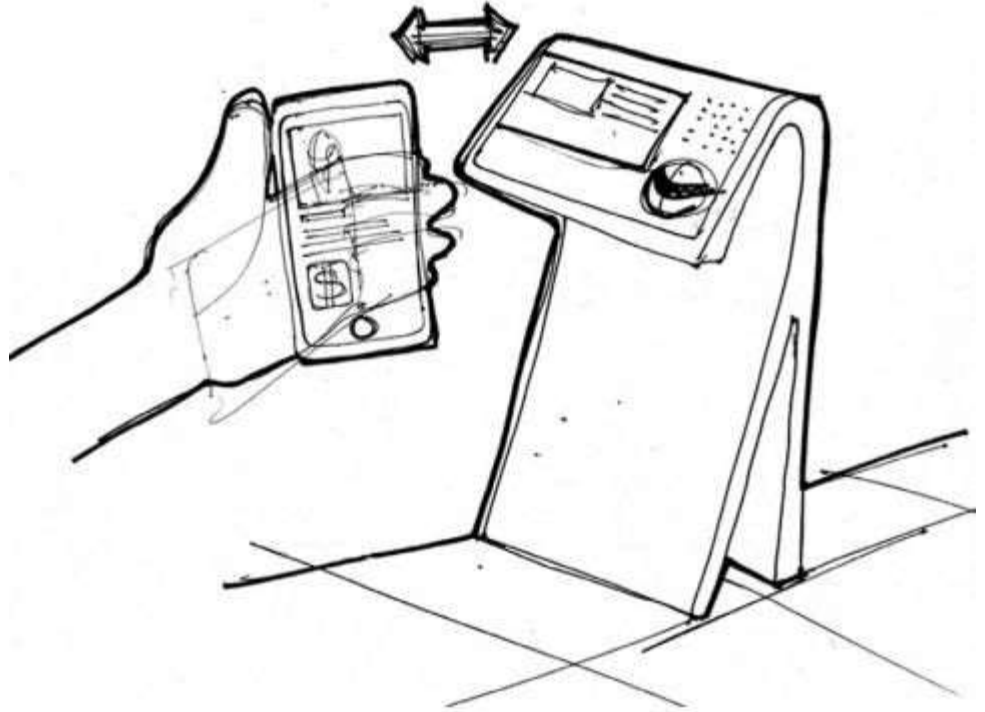
Early conceptual design
ideas from the ecological
perspective



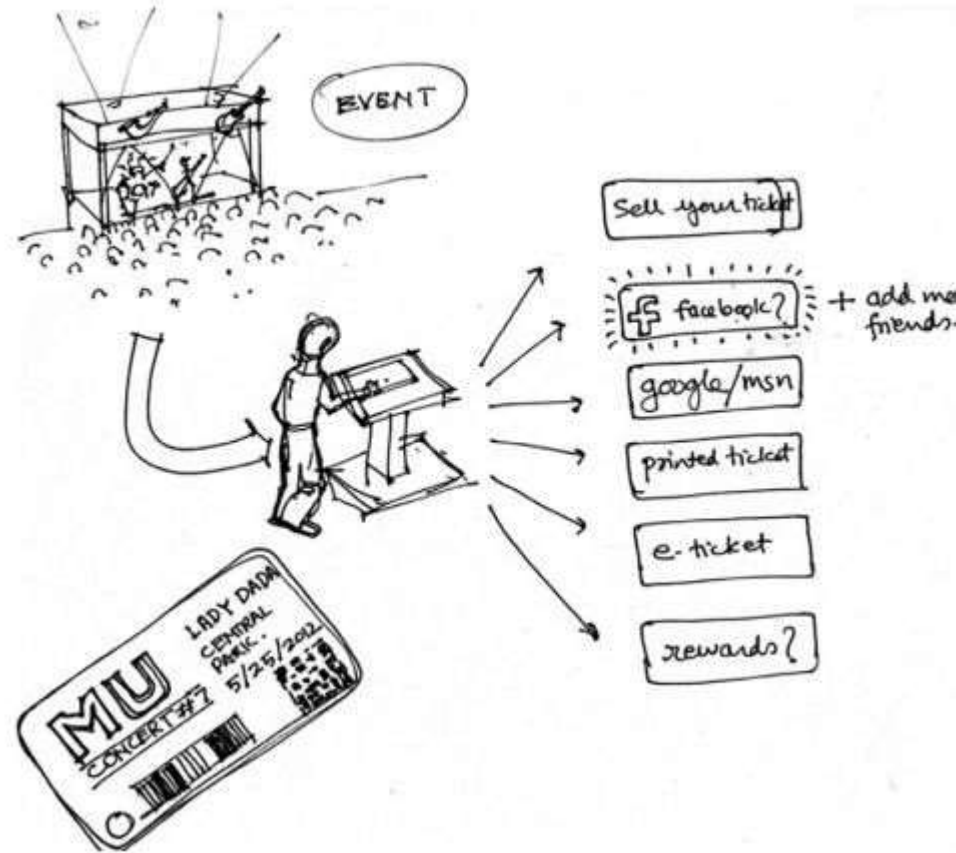
Ecological conceptual
design ideas focusing on a
feature for a smart ticket to
guide users to seating



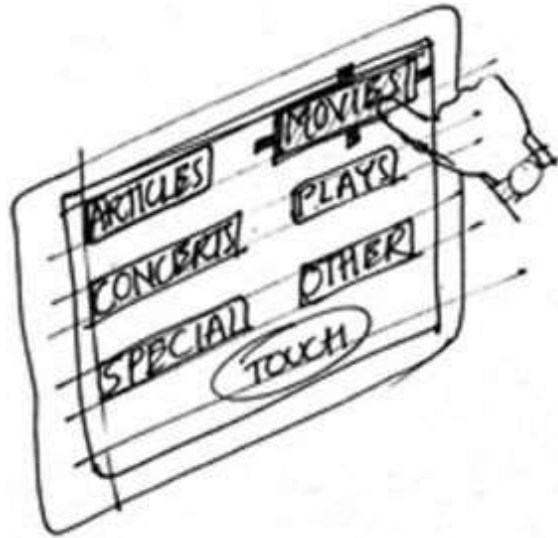
Ecological conceptual
design ideas focusing on a
feature showing
communication connection
with a smartphone



Ecological conceptual
design ideas focusing
on the features for
communicating and social
networking



Part of a conceptual design
in the interaction
perspective



STORYBOARDS

What Are Storyboards?

A **storyboard** is a sequence of visual frames that illustrate **how a user interacts with a system**.

- Similar to a **comic strip**, each frame represents a key moment in a user's journey.
- Storyboards focus on **the flow of interaction**, rather than technical details.
- They help designers **anticipate user actions, challenges, and needs**.

Why Use Storyboards?

- **Brings design ideas to life** by visually demonstrating interactions.
- **Helps identify usability issues** before development begins.
- **Communicates design concepts** to stakeholders and team members.
- **Ensures user-centered design** by focusing on real-world user scenarios.

Making Storyboards to Cover All Design Perspectives

Storyboards should **address different perspectives** to create a holistic design vision.

What to Include in a Storyboard?

- **Hand-drawn sketches** or digital visuals with short descriptions.
- **User's work environment** (e.g., interacting with a phone, desktop, kiosk).
- **Screens, devices, and UI elements** users interact with.
- **User thoughts, actions, and decisions** ("thought bubbles").
- **Physical actions** (e.g., gestures, pressing buttons).
- **Extra-system activities** (e.g., talking to a friend, receiving an email).

Storyboards in the Three Design Perspectives

1. Ecological Perspective: System in Its Environment

- Shows how the system fits into the **user's work and social context**.
- Example: **A handheld device used while waiting at an airport.**
- Focuses on **how the system interacts with other elements in the environment.**

2. Interaction Perspective: User Actions and Interface

- Visualizes **user actions, UI screens, transitions, and reactions.**
- Example: **A ticket kiosk user navigating through seat selection and payment screens.**
- Focuses on **task sequences, system responses, and user inputs.**

3. Emotional Perspective: User Experience and Feelings

- Illustrates **emotions, aesthetics, and engagement factors.**
- Example: **A user's excitement while interacting with a beautifully designed app.**
- Captures **moments of delight, frustration, or satisfaction.**

Example: Storyboards for a Ticket Kiosk System

Ecological Perspective: Ticket Kiosk in Context

- User **approaches the kiosk** at an event venue.
- The system **detects the user's presence** and personalizes options.
- Other people are **waiting in line**, creating a **real-world context**.

Interaction Perspective: User's Step-by-Step Actions

1. User **selects an event** from the main menu.
2. System **displays seat availability** with a seating chart.
3. User **chooses a seat**, enters payment details, and **receives a confirmation**.

Emotional Perspective: Enhancing User Experience

- A **welcoming message** makes the user feel **engaged**.
- The interface **uses animations and color feedback** to create an **immersive experience**.
- The system plays **background music or sounds** to enhance the ambiance.

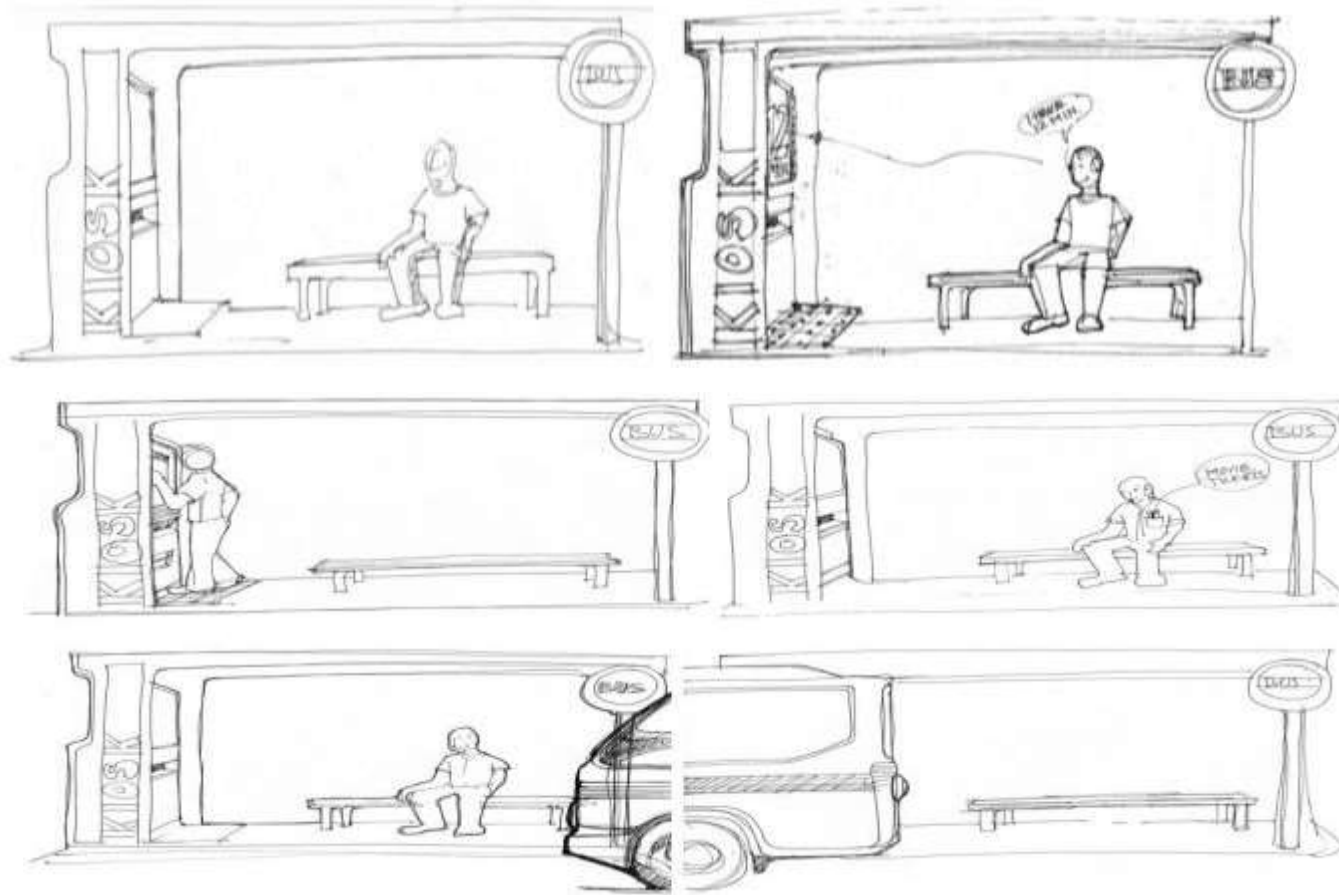
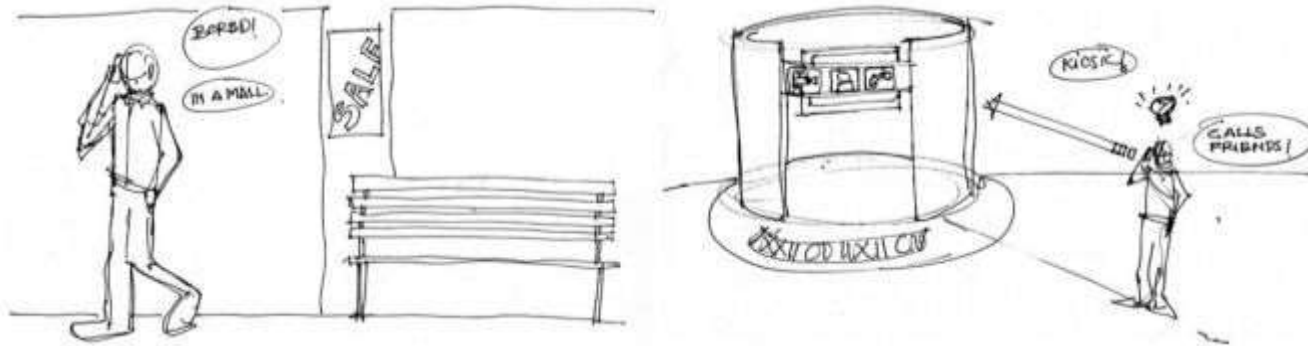
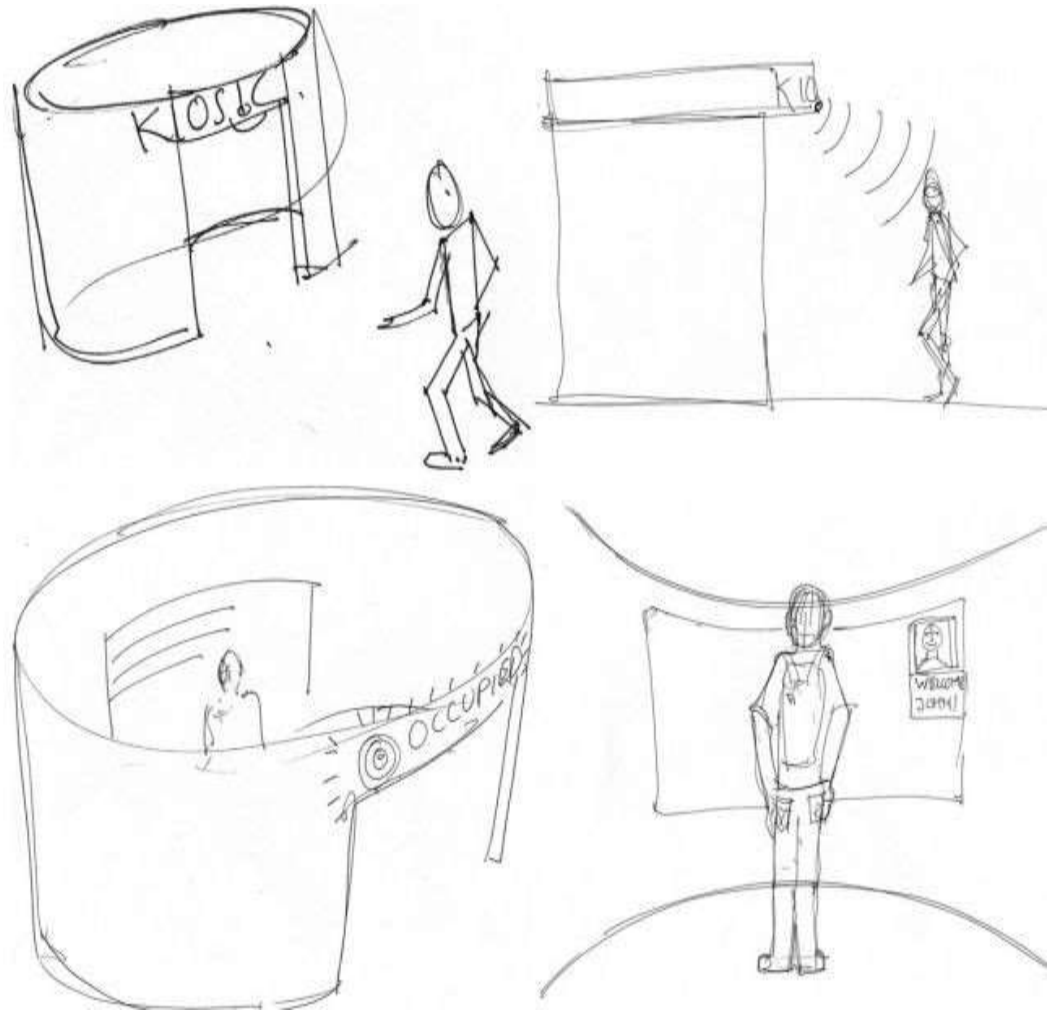


Fig: Example of a sequence of sketches as a storyboard in the ecological perspective (Ticket Kiosk System)

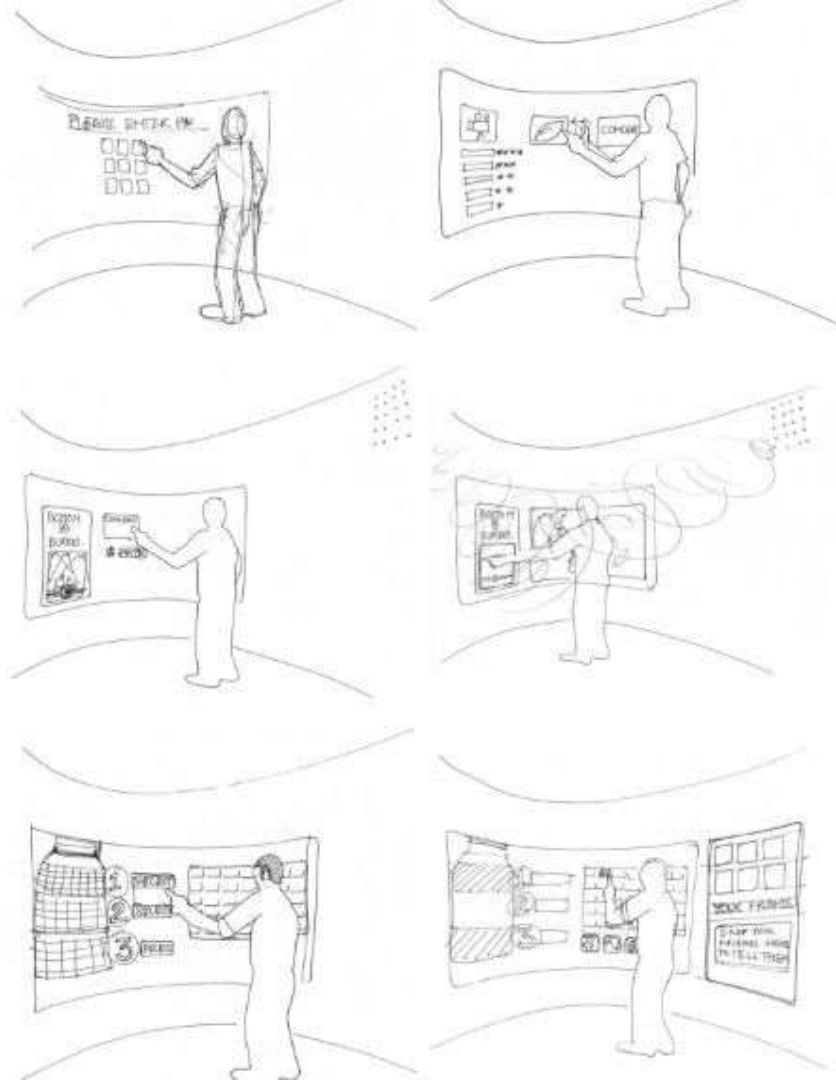


Part of a different Ticket Kiosk System storyboard in the ecological perspective

Sample sketches for a similar
concert ticket purchase
storyboard in the interaction
perspective



Sample sketches for a similar
concert ticket purchase storyboard in
the interaction perspective (*Contd.*)



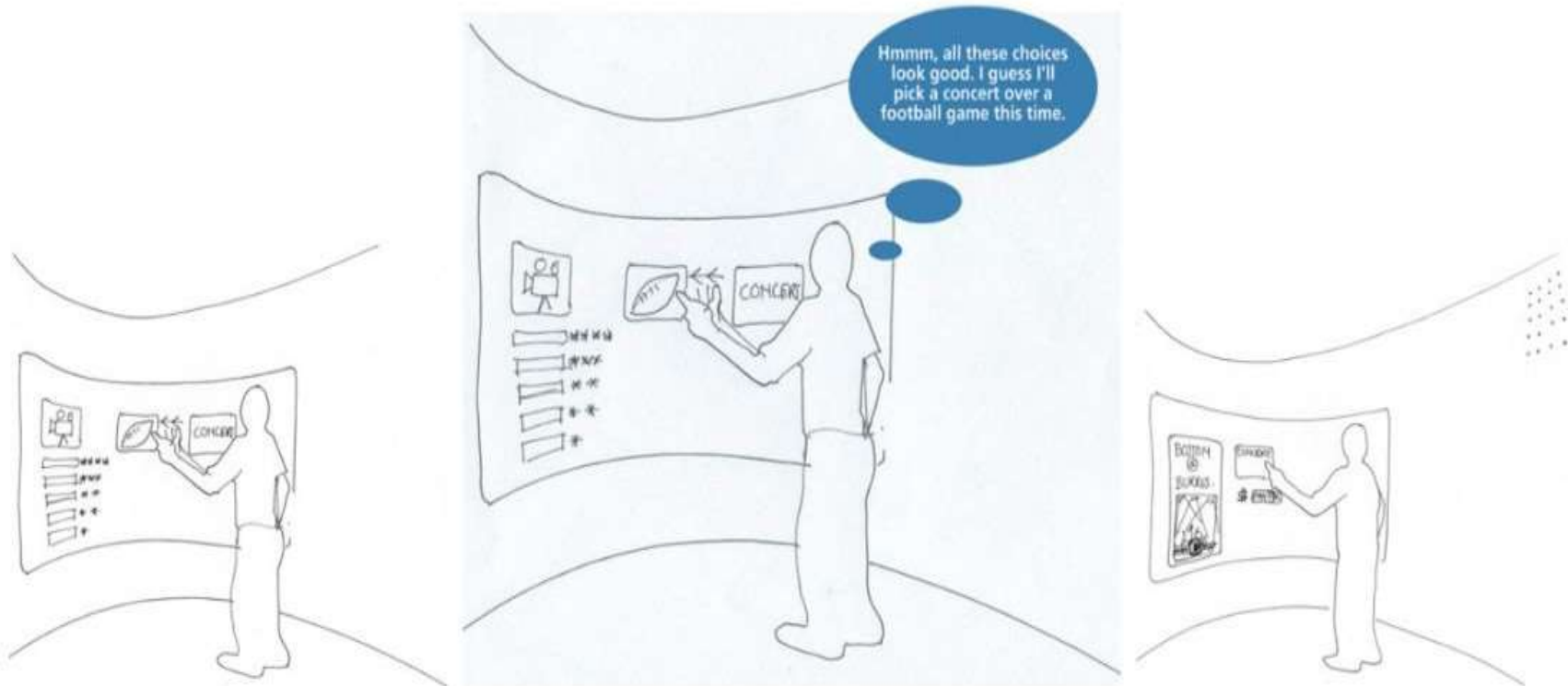
Importance of Between-Frame Transitions

Storyboards typically show **static frames**, but **user experience is about transitions** between those frames.

- **Transitions define how interactions feel**, such as:
 - Screen fades, animations, or button presses.
 - How quickly a response appears.
 - What happens **between** selecting an option and seeing the next screen.
- Example: A **storyboard for a mobile app** should show:
 - User **scrolling through options** (gesture).
 - A **loading animation** while the system processes the request.
 - A **smooth transition** to the next screen.

Why Are Transitions Important?

- Transitions **shape the user's perception of the system.**
- Poorly designed transitions create **confusion and frustration.**
- Good transitions improve **usability and satisfaction.**



Storyboard transition frame with thought bubble explaining state change

DESIGN INFLUENCING USER BEHAVIOR

Slanty Design: Encouraging or Restricting Certain User Behaviors

Beale (2007) introduced the concept of **slanty design**, which aims to influence user behavior **by modifying usability in specific ways**. Instead of making everything as easy as possible, designers can introduce constraints to **nudge users toward better actions**.

Example: Slanted Reading Desks in Libraries

- These desks have a **sloped surface**, making it **difficult to place food or drinks** on them.
- This discourages users from eating at the desk, thus **protecting books and documents**.

Example: Airport Baggage Carousels

- People tend to **crowd the baggage carousel**, making it hard for others to retrieve their luggage.
- A **sloped floor** around the carousel **pushes people away**, forcing them to stand further back and reducing congestion.

How Slanty Design Works

- **Encourages or discourages certain actions** without explicit rules or enforcement.
- **Balances usability** between **individual users** and the **broader system or community**.

Design Influencing User Behavior in Everyday Life

Interaction design can shape user behavior beyond usability—such as promoting environmental awareness, safety, and efficiency.

Example: Amazon Kindle and Reading Habits

- The Kindle makes books **instantly available** and **portable**, increasing reading frequency.
- Users **adapt to digital reading**, which influences how they consume books.

Example: Green Design and Energy Conservation

- Devices that **turn off automatically** when not in use encourage users to **save energy**.
- Example: **Motion-activated lights** that shut off when no movement is detected.

DESIGN FOR EMBODIED INTERACTION

Embodied interaction is about designing systems that allow users to interact with technology **naturally, using physical actions, gestures, and movement.**

- Unlike traditional interfaces that rely solely on screens, embodied interaction **engages the user's body, movement, and senses** to enhance usability and experience.
- Instead of limiting interaction to **button clicks and screen taps**, embodied interaction **incorporates the physical world**, making interactions **more natural and intuitive.**

Examples of Embodied Interaction

1. Physical Assembly vs. Mental Processing

- A person assembling furniture **understands better when handling pieces** rather than just reading instructions.

2. Physical Mockups in Design

- Designers use **physical models** to prototype products, allowing for **better interaction and ideation**.

3. Gesture-Based Controls

- Virtual Reality (VR) systems use **hand movements** to manipulate objects, creating **a more immersive experience**.

Designing for Embodied Interaction

To design for embodied interaction, consider:

- ✓ **Hands, eyes, and body movement** → Example: Hand-gesture navigation.
- ✓ **Physical engagement with the environment** → Example: Smart home sensors reacting to presence.
- ✓ **Leveraging innate human traits** → Example: VR interactions that mimic real-world object manipulation.

Embodied vs. Tangible Interaction

Embodied Interaction	Tangible Interaction
Physical movement & body engagement	Interaction with physical objects
Gesture-based controls (e.g., waving a hand to navigate a menu)	Touch-sensitive physical buttons on a device
Cognition is shaped by physical actions	Interaction is embedded in real-world objects

Example: Scrabble Flash Cubes (Hasbro) – Embodied & Tangible Interaction

- A modern **digital Scrabble game** where users **physically manipulate letter tiles**.
- Tiles **recognize their arrangement**, lighting up when they form a valid word.
- **Why It's Embodied Interaction:**
 - Players **move and arrange the tiles with their hands**, rather than using a touchscreen.
 - The game mimics **natural interaction** with objects, making it more intuitive.

The Scrabble
Flash Cube
game.



UBIQUITOUS AND SITUATED INTERACTION

Ubiquitous, Embedded, and Ambient Computing

What Is Ubiquitous Computing?

- **Ubiquitous computing** (or **pervasive computing**) refers to the concept of **technology being present everywhere**, embedded in objects and environments rather than being confined to a single device.
- Users do not have to **consciously interact** with a system for it to function—it **operates passively**, providing information or services as needed.

Example:

- **Smart home automation:** A house that **adjusts lighting, temperature, and security automatically** based on the presence of its occupants.
- **Wearables:** Smartwatches or fitness bands that **track heart rate and activity without requiring user input**.

What Is Embedded Computing?

- **Embedded computing** refers to **technology integrated into physical objects**, making them "smart."
- These objects can **process information, communicate with other devices, and make automated decisions.**

Example:

- **RFID tags** in supermarkets allow stores to **track inventory** without manual scanning.
- **Smart refrigerators** monitor food expiration dates and suggest recipes based on available ingredients.

What Is Ambient Computing?

- **Ambient computing** is the next step in ubiquitous computing, where **technology blends seamlessly into the environment** and adapts automatically to users' needs.
- It involves a **network of interconnected devices** working together **without requiring explicit input from the user**.

Example:

- A smart office system that **automatically adjusts lighting, temperature, and workspace settings** when an employee enters a room.
- **Amazon Echo (Alexa) and Google Home**, which **passively listen and respond to voice commands**, integrating with smart home systems.

Situated Awareness and Situated Action

- **Situated awareness** refers to a system's ability to **detect and interpret its surroundings** in real-time.
- Instead of requiring **explicit input**, the system **understands its environment** and **adjusts its behavior accordingly**.

Example:

- **A smartphone that automatically switches to silent mode** when it detects that the user is in a movie theater.
- **Location-based notifications**, such as **Google Maps suggesting restaurants** when a user enters a new city.

- **Situated action** refers to interactions that are **context-dependent and dynamically adapted** rather than pre-planned.
- Users interact **spontaneously**, and the system **adjusts its responses based on real-time factors**.

Example:

- **Voice assistants (like Siri or Alexa)** provide **different responses based on user context**, such as recommending an umbrella if it's about to rain.
- **A smart assistant suggests activities** based on weather, traffic, or the user's schedule.

PROTOTYPING

- Prototyping is a way to create a **design representation**, which can overlap with the actual design process.
- While it might appear that prototyping happens at a specific stage in the lifecycle, it is not limited to a particular phase. Prototyping often occurs iteratively alongside other activities.

The Dilemma in Software Development

- Traditional development methods, such as the **waterfall model**, demand heavy investments in time and resources. These linear methods often lock in design decisions early without any means to test them until late in the process.
- This approach risks delivering systems with **wrong requirements**, **imbalanced functionality**, or **poor user experience**, leading to user dissatisfaction.

The Solution: Prototyping

- Prototypes act as **early, inexpensive models** of a system, allowing teams to:
 - Test designs with users **before full implementation**.
 - Evaluate, refine, and address design issues iteratively.
- Prototyping supports the iterative lifecycle by providing something tangible to test and refine early in the design process.

Universality of Prototyping

- Prototyping is a **timeless concept** found across disciplines:
 - **Automobile designers** build mockups.
 - **Architects and sculptors** create models.
 - **Circuit designers** use breadboards.
 - **Filmmakers**, like Alfred Hitchcock, test storylines with audience reactions.
 - Historical figures like **Leonardo da Vinci** and **Thomas Edison** leveraged prototypes for experimentation and refinement.

DEPTH AND BREADTH OF A PROTOTYPE

Prototyping choices often involve decisions about:

- **Breadth:** Covering a wide range of features but with limited detail.
- **Depth:** Focusing on specific features with significant detail.
- Prototypes can be classified based on the depth and breadth they are covering:
 - a. Horizontal Prototypes
 - b. Vertical Prototypes
 - c. “T” Prototypes
 - d. Local Prototypes

a) Horizontal Prototypes:

- Horizontal prototypes are breadth based.
- Focuses on covering a wide range of features but lack depth in functionality.
- Suitable for providing an **overview** of the system.
- Use cases:
 - Demonstrating the concept to stakeholders.
 - Exploring which features users find most important.
 - Gathering feedback on the breadth of functionality.
- Limitation: They do not support complete workflows or realistic user experience evaluations due to their shallow depth.
- **Example:** Imagine a prototype of an **e-commerce website**:
 - You can click on **Home**, **Products**, **Cart**, and **Profile** pages, but these pages don't have detailed functions like adding items to the cart or making payments.
 - It gives a general idea of the website structure.

b) Vertical Prototypes:

- Vertical prototypes are depth based.
- Focuses deeply on a specific feature, showing detailed functionality.
- Use cases:
 - Testing isolated interaction workflows.
 - Evaluating the usability of specific components in detail.
- Limitation: They cover only a small part of the system, missing the overall picture.
- **Example:** For the **e-commerce website**:
 - The prototype focuses deeply on the **checkout process**, allowing you to select items, enter payment details, and confirm the purchase, even if the rest of the site isn't functional.
 - This allows for realistic user testing of the message workflow, even if other parts of the system aren't functional

c) "T" Prototypes

- A **"T" Prototype** combines the advantages of both horizontal and vertical prototypes:
 - Broad coverage of features (like horizontal prototypes).
 - In-depth development for a few key features (like vertical prototypes).
- Use case:
 - Effective for system evaluation, providing both an overview and detailed exploration of critical features.
 - Often used as a next step after creating a horizontal prototype, evolving into greater depth over time.
- **Example:** In the **e-commerce website**:
 - You can navigate through all main pages (like a horizontal prototype), but the **product page and checkout process** have full functionality (like a vertical prototype).
 - This helps in getting a complete picture while deeply testing critical functions.

d) Local Prototypes

- A **Local Prototype** focuses on a very small and specific part of the design, addressing isolated interaction issues (e.g., the appearance of an icon or the wording of a message).
- Characteristics:
 - Limited in both breadth and depth.
 - Used to resolve design disagreements or test alternatives for minor elements.
 - Short lifespan, as they are often replaced once a decision is made.
- Use case:
 - Breaking impasses in design discussions by allowing user input to guide decisions.

Example:

- Suppose the design team for the **e-commerce website** is debating the layout of the "Save" button in the screen. A local prototype would create different button designs to gather user feedback on—color, size, or placement, etc.—without changing anything else on the page.
- This helps make quick design decisions without building large prototypes.

FIDELITY OF PROTOTYPES

- Fidelity refers to **how closely a prototype resembles the final product** in terms of appearance, functionality, and interaction.
- It reflects how "finished" a prototype looks, though it doesn't necessarily mean the underlying functionality is complete.
- Types of Prototype Fidelity”
 - Low-Fidelity Prototypes
 - Medium-Fidelity Prototypes
 - High-Fidelity Prototypes

Low-Fidelity Prototypes

Characteristics:

- Simple, rough representations of the design.
- Usually made using paper, sketches, or basic tools.
- Do not include detailed look, feel, or behavior.

Advantages:

- Easy to create, modify, and iterate upon.
- Inexpensive and quick to produce.
- Effective for brainstorming and early design exploration.
- Encourages team collaboration and creativity.
- Focuses feedback on **high-level design issues** like workflows and functionality, rather than aesthetics.

Misconceptions:

- Some teams may initially dismiss low-fidelity prototypes as "amateurish," but studies have shown users take them seriously and provide valuable feedback.

Use Cases:

- Early design stages to test concepts and workflows.
- Quickly gathering user feedback.

Medium-Fidelity Prototypes

Characteristics:

- More polished than low-fidelity prototypes but not fully realistic.
- Often computer-based, such as wireframes (basic visual guide or blueprint that represents the structure and layout of a digital product) or simple digital mockups.
- Demonstrate **layout, navigation, and basic interactions**.

Advantages:

- Balance between detail and speed of production.
- Suitable for testing interaction design and navigation flows.
- Useful for communicating with stakeholders and development teams.

Use Cases:

- Intermediate design stages when you need more detailed feedback but don't want to commit to final design details.

High-Fidelity Prototypes

Characteristics:

- Detailed, realistic representations of the final product.
- Include specific appearance, behavior, and interaction details.
- May include functional back-end components (e.g., working databases or APIs).

Advantages:

- Allow users to evaluate the design as if it were the real product.
- Useful for refining detailed design elements like color schemes, animations, and error handling.
- Serve as effective tools for stakeholder presentations, marketing demos, or securing investment.

Disadvantages:

- Expensive and time-consuming to create.
- Difficult to modify, especially if design changes are required.

Use Cases:

- Later stages of design to finalize appearance and interactions.
- Advanced evaluations and usability testing.

When to Use Each Level of Fidelity

- **Low-Fidelity:**

- Early stages of design for ideation and exploring workflows.
- When you need quick iterations and early feedback.

- **Medium-Fidelity:**

- Intermediate stages to flesh out navigation, layouts, and interaction flows.
- When communicating with stakeholders or testing basic usability.

- **High-Fidelity:**

- Later stages to finalize design details and validate the overall user experience.
- For presenting to clients or stakeholders who require a realistic preview of the product.

INTERACTIVITY OF PROTOTYPES

- Interactivity in a prototype refers to how it responds to user actions, such as clicking, typing, or navigating through screens.
- The amount of interactivity is often linked to the **level of fidelity**: higher interactivity generally requires higher-fidelity prototypes.
- Interactivity helps simulate real-world usage, making it easier to evaluate user behavior and refine designs.
- Types of Interactive Prototypes:
 - Scripted and “Click-Through” Prototypes
 - A Fully Programmed Prototype
 - “Wizard of Oz” Prototypes
 - Physical Mockups
 - Paper-in-Device Mockup Prototypes
 - Animated Prototypes

1. Scripted and “Click-Through” Prototypes

Scripted Prototypes:

- Programmed using simple scripting languages to add basic behaviors.
- Create "live-action storyboards" that respond in a limited way to user actions.
- Typically low- or medium-fidelity.
- Examples: Moving between predefined screens or animations triggered by specific actions.

Click-Through Prototypes:

- Contain active links or buttons that allow users to navigate through screens.
- Often created using wireframes or mockups with clickable elements.
- Limited interactivity: clicking moves the user to the next screen or section but lacks complex functionality.
- Examples: Clicking a "Next" button to navigate through screens in a user flow.

2. Fully Programmed Prototypes

Characteristics:

- High-fidelity prototypes with a significant level of functionality.
- Simulate the complete behavior of the system, often including back-end processes (e.g., a working database).
- Expensive and time-consuming to create but provide a highly realistic experience.

Use Case:

- Used in large projects where stakeholders or users need to experience the product in detail before full implementation.
- Example: An air traffic control system prototype built as part of a large-scale project proposal.

3. “Wizard of Oz” Prototypes

- A technique where the user interacts with what appears to be an automated system, but a human is controlling the responses behind the scenes.
- **Setup:** Two computers are connected: one for the user and one for a hidden evaluator who manually simulates system responses.
- **Advantages:**
 - Simulates highly interactive systems, even those requiring complex or adaptive behavior (e.g., AI-driven applications), without full implementation.
 - Useful for exploring user behavior in uncertain or evolving scenarios.
- **Example:**
 - Early user interface designs for an email system where a human simulated system responses to unknown commands, allowing iterative improvement of the design.

4. Physical Mockups

- **Physical mockups** are prototypes created for systems or products with a physical component, such as handheld devices, kiosks, or other hardware.
- **Approach:**
 - Can involve materials like cardboard, wood, or even 3D-printed components to mimic the form factor and interaction.
 - Enhances realism by allowing users to interact with the physical design.
- **Example:**
 - A cardboard prototype of a mobile device used to test ergonomics and button placement.

5. Paper-in-Device Mockup Prototypes

- A variation of low-fidelity prototypes where paper screens are loaded into a physical device (e.g., a phone) to simulate digital interaction.
- **Method:**
 - Paper screens are scanned and displayed as a sequence of images that users navigate through using the device's existing touch or gesture controls.
- **Advantages:**
 - Combines the flexibility of paper prototyping with the interactivity of a digital device.
 - Agile, cost-effective, and useful for testing mobile interfaces.

6. Animated Prototypes

- Non-interactive but dynamic prototypes created using animations or video to simulate the flow of interactions.

Purpose:

- Useful for presenting concepts, visualizing new interaction designs, or communicating ideas to stakeholders.
- Example: An animated walkthrough of a new app interface showing navigation flows and transitions.

Choosing the Right Level of Interactivity

- The amount of interactivity required depends on the **design stage** and **evaluation goals**:
 - **Low Interactivity**: For early exploration and brainstorming.
 - **Medium Interactivity**: To evaluate navigation flows, layouts, and simple behaviors.
 - **High Interactivity**: For advanced usability testing, final design refinement, or stakeholder presentations.
- Interactivity increases the realism of the prototype, which can improve user feedback but also requires more effort and resources.

CHOOSING THE RIGHT BREADTH, DEPTH, LEVEL OF FIDELITY, AND AMOUNT OF INTERACTIVITY

Two Key Factors in Choosing Prototyping Approaches

1. The Stage of Progress in the Project:
 - Early, intermediate, or final stages of design have different prototyping needs.
2. The Design Perspective Being Addressed:
 - Prototyping needs vary depending on whether you are focusing on ecological context, interaction design, or emotional impact.

Using the Right Level of Fidelity for the Current Stage of Progress

Choosing the right audience: Low-fidelity prototypes are mainly for internal team use, while high-fidelity ones are for external stakeholders. Without explanation, low-fidelity prototypes may seem unpolished and create misunderstandings.

A progression of increasing fidelity:

- **Early-stage:** Low-fidelity prototypes (sketches, wireframes) help explore ideas and overall system concepts.
- **Mid-stage:** Medium-fidelity prototypes refine interaction design and navigation.
- **Late-stage:** High-fidelity prototypes focus on final visual design, usability testing, and detailed interactions.

Using the Right Level of Fidelity for the Design Perspective Being Addressed

Different design perspectives require different types of prototypes:

1. Ecological Perspective:

- Explores how the system fits into its environment.
- Uses low-fidelity concept maps and sketches to visualize system interaction with external factors.
- Example: IBM's Olympic Message System used "Wizard of Oz" testing with physical kiosks.

2. Interaction Perspective:

- Focuses on how users interact with the system.
- Begins with **low-fidelity prototypes** (sketches, paper prototypes) to explore different workflows.
- Moves to **medium-fidelity wireframes** for layout and functionality.
- Ends with **high-fidelity prototypes** to test detailed interactions.

3. Emotional Perspective:

- Evaluates emotional impact, aesthetics, and user satisfaction.
- **High-fidelity prototypes** are usually required.
- Physical mockups are important for tangible devices like mobile phones or kiosks.

Summary of the
uses for
various levels of
fidelity and types
of prototypes

Kind of Iteration	Purpose	Types of Prototypes
Ideation and sketching	To support exploring ideas, brainstorming, and discussion (so design details are inappropriate)	Sketches, fast and disposable mockups, ultralow fidelity
Conceptual design	To support exploration and creation of conceptual design, the high-level system structure, and the overall interaction metaphor	Evolution from hand-drawn paper, computer-printed paper, low-fidelity wireframes, high-fidelity wireframes, to pixel-perfect interactive mockups (to communicate with customer)
Intermediate design	To support interaction design for tasks and task threads	Evolution from paper to wireframes
Detailed design	Support for deciding navigation details, screen design and layout, including pixel-perfect visual comps complete specification for look and feel of the "skin"	Detailed wireframes and/or pixel-perfect interactive mockups
Design refinement	To support evaluation to refine a chosen design by finding and removing as many UX problems as possible	Medium to high fidelity, lots of design detail, possibly a programmed prototype

Managing Risk and Cost within Stages of Progress and within Design Perspectives

- **Low-fidelity prototypes** are inexpensive and allow quick iterations.
- **High-fidelity prototypes** take more time and resources, so they should only be introduced when interaction flows are solidified.

Type of Prototype	"Strength"	When in Lifecycle to Apply "Strength"	Cost to Fix Appearance	Cost to Fix Sequencing
Low fidelity (e.g., paper)	Flexibility; easy to change sequencing, overall behavior	Early	Almost none	Low
High fidelity (e.g., computer)	Fidelity of appearance	Later	Intermediate	High

Fig: Summary of comparison of low-fidelity and high-fidelity prototypes

PAPER PROTOTYPES

- **Paper prototyping** is a low-cost, fast, and flexible way to create and test design ideas before committing to software development.
- Paper prototypes are often the first step in **prototyping an interaction design**, allowing teams to **quickly iterate and refine concepts**.
- Paper prototypes are used to demonstrate the overall interaction design and screen flow in early stages.
- These prototypes do not have functionality but show **static screen layouts and sequencing** for discussion.
- The goal is to create and test designs in **hours or days**, rather than weeks.

Hand-Drawn Paper Prototypes

- Screens and interface elements are drawn by hand on paper.
- A human "executor" (acting as the system) manually changes screens when users "interact" with the prototype.

Computer-Printed Paper Prototypes

- Paper prototypes, with user interface objects and text on paper printed via a computer, are essentially the same as hand-drawn paper prototypes, except slightly higher fidelity in appearance.
- These are created using software like **Microsoft Visio, OmniGraffle, or even Excel.**

Is Paper Just a Stopgap Medium?

- **No, paper prototyping is a legitimate technique.**
- **Advantages:**
 - Faster and easier to modify than software prototypes.
 - Encourages creativity and focuses on **design, not programming**.
 - Avoids spending too much time coding before finalizing design choices.

Why Not Just Program a Low-Fidelity Prototype?

- **Paper prototypes are always available for evaluation**, while software prototypes require debugging and version updates.
- **Easier comparison:** Paper allows **multiple screens to be laid out side by side**, making it easy to compare different interactions.
- **More flexible:** Changes can be made instantly without reprogramming.

How to Make an Effective Paper Prototype

1. Start by setting a realistic deadline.
2. Gather a set of paper prototyping materials.
3. Work fast and do not color within the lines.
4. Draw on everything you have worked on so far for the design.
5. Make an easel to register (align) your screen and user interface object sheets of paper and plastic.
6. Make underlying paper foundation “screens.”
7. Use paper cutouts taped onto full-size plastic “interaction sheets” for all moving parts.
8. Be creative.



Fig1: Foam-core board paper prototype easel with “stops” to align the interaction sheets.



Fig2: Another style of “stops” on a foam-core board paper prototype easel.



Fig3: Underlying paper foundation “screen.”

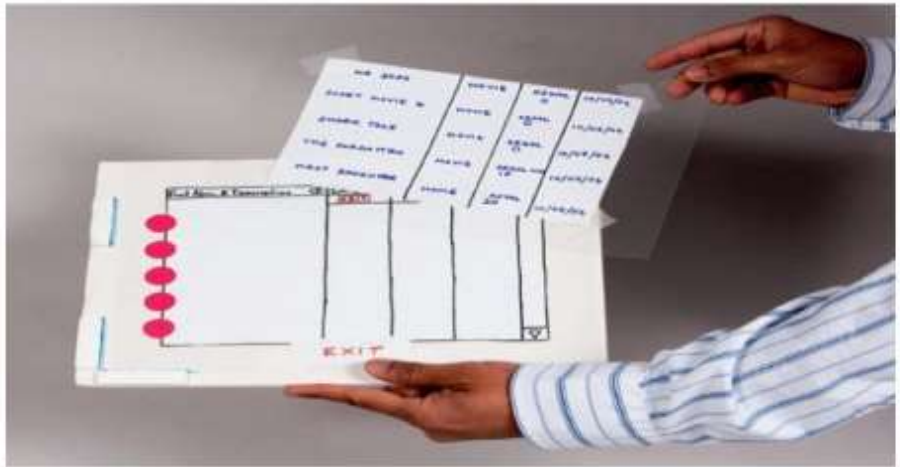


Fig4: Paper cutouts taped to full-size plastic for moving parts.

10. Do not write or mark on plastic interaction sheets.
11. Make highlights on plastic with “handles” for holding during prototype execution.
12. Make your interaction sheets highly modular by including only a small amount on each one.
13. Get modularity by thinking about whatever needs to appear by itself.
14. Do lots of sketching and storyboarding before making interaction sheets.
15. Use every stratagem for minimizing work and time.
16. Reuse at every level.

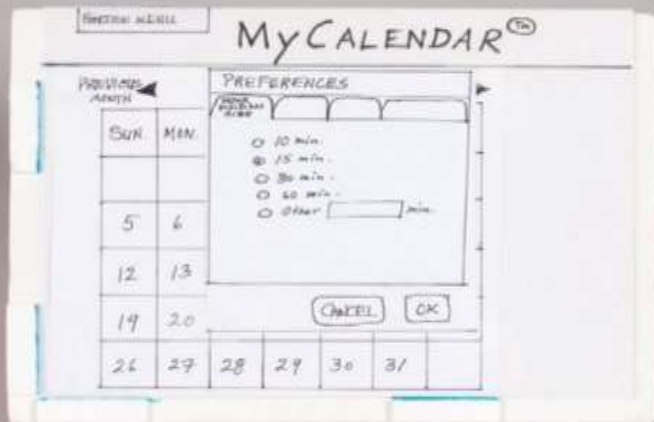


Fig5: A “Preferences” dialogue box taped to plastic and aligned in easel.



Fig6: Pull-down menu on a tape “hinge.”

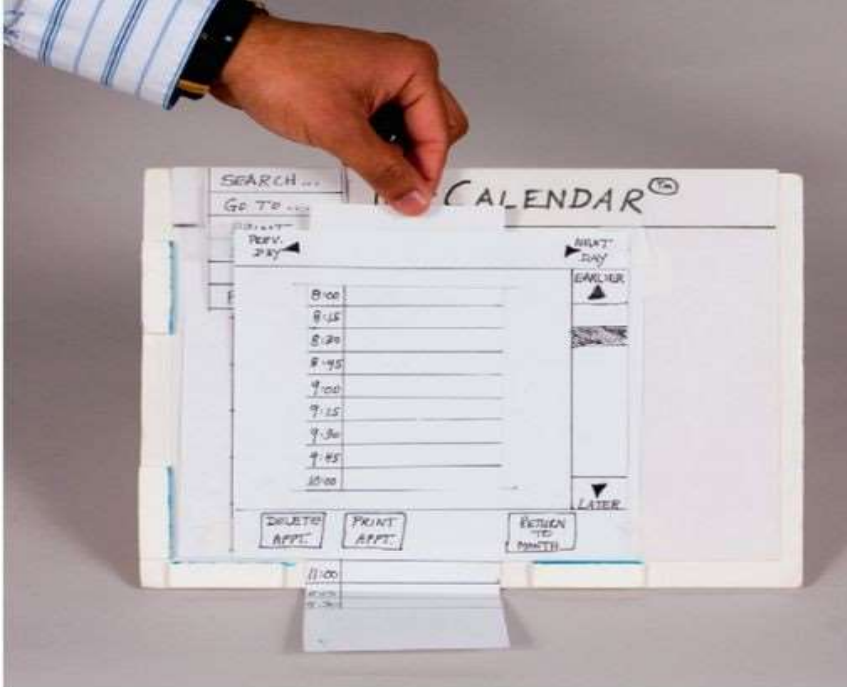


Fig7: Paper sliding through a slit for scrolling.



Fig8: Selection highlight on plastic with a long handle.

17. Cut corners when it does not hurt things.
18. Make the prototype support key tasks.
19. Make a “this feature not yet implemented” message.
20. Include “decoy” user interface objects.
21. Accommodate data value entry by users.
22. Create a way to manage complex task threads.
23. Pilot test thoroughly.

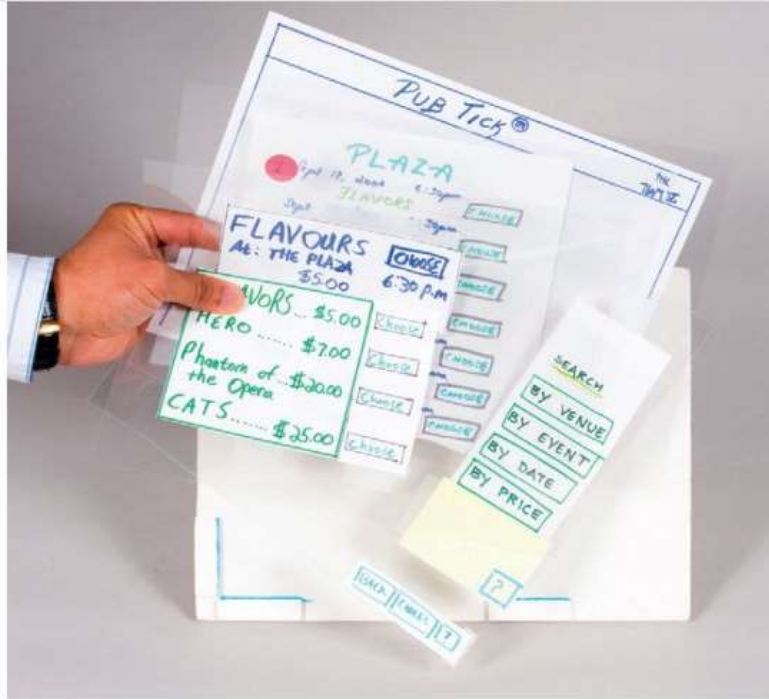


Fig9: Lots of pieces of dialogue as paper cutouts aligned on plastic sheets.

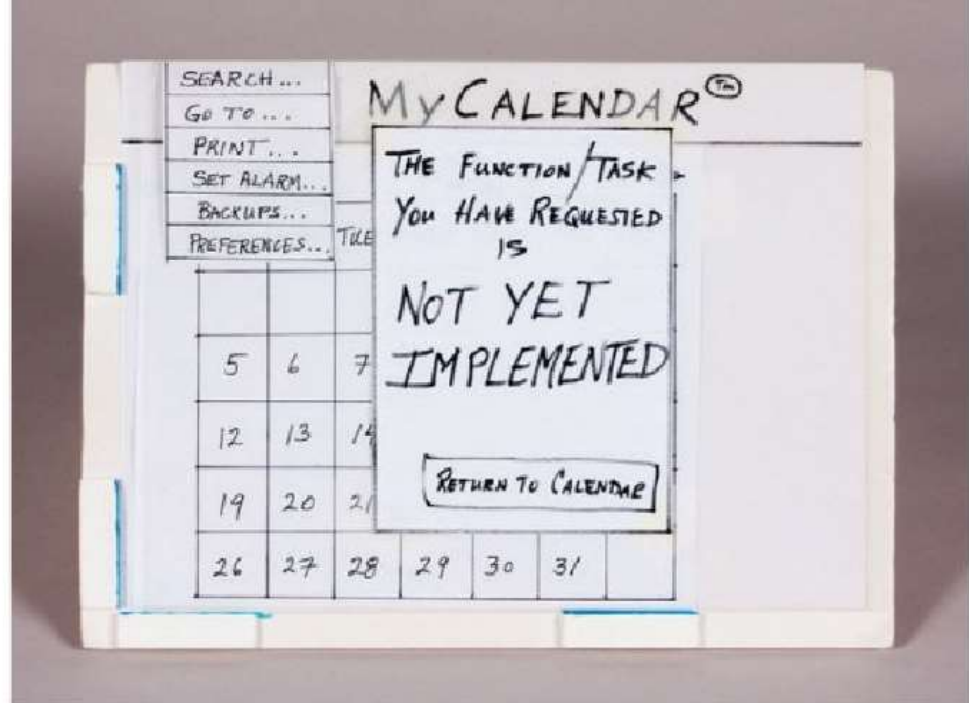


Fig10: “Not yet implemented” message.

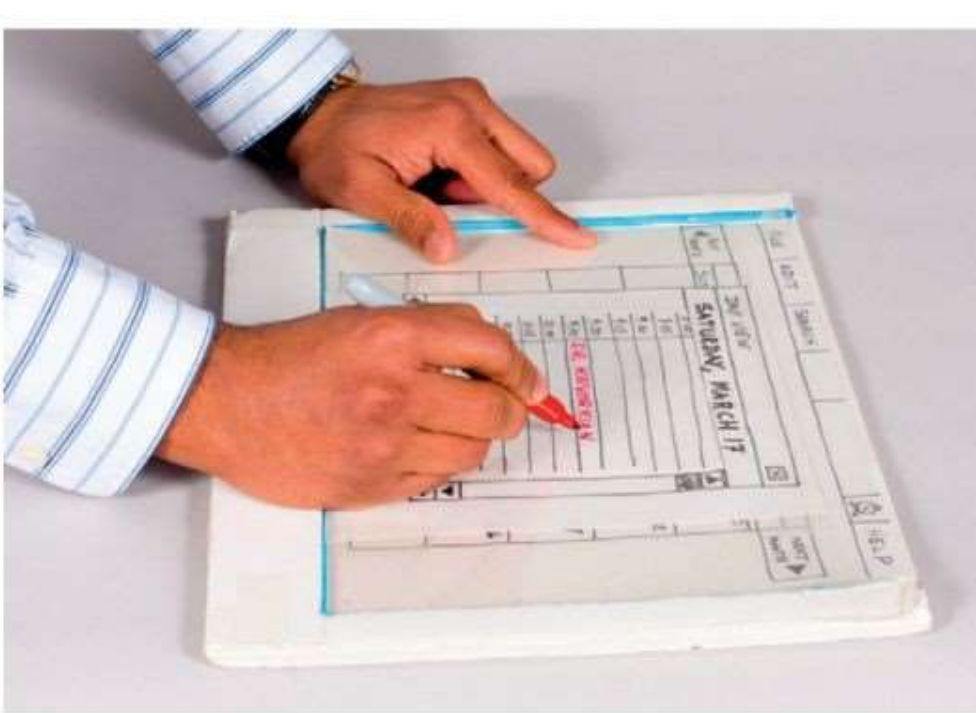


Fig11: Data entry on clear plastic overlay sheet.

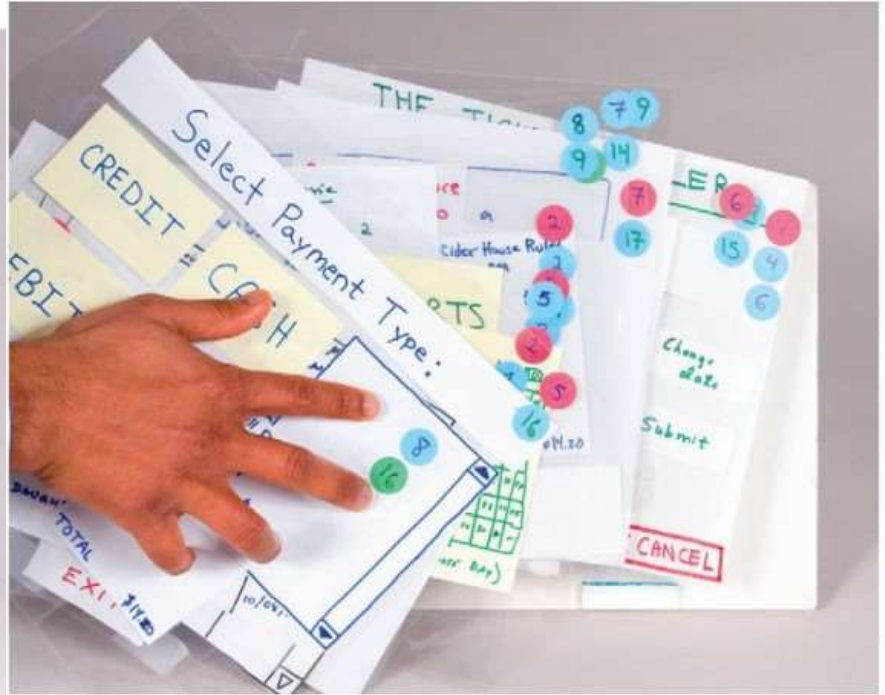


Fig12: Adhesive-backed circles for color coding task threads on prototype pieces.

ADVANTAGES OF AND CAUTIONS ABOUT USING PROTOTYPES

Advantages of Prototyping

- Offer concrete baseline for communication between users and designers
- Provide conversational “prop” to support communication of concepts not easily conveyed verbally
- Allow users to “take the design for a spin” (who would buy a car without taking it for test drive or buy a stereo system without first listening to it?)
- Give project visibility and buy-in within customer and developer organizations
- Encourage early user participation and involvement
- Give impression that design is easy to change because a prototype is obviously not finished
- Afford designers immediate observation of user performance and consequences of design decisions
- Help sell management an idea for new product
- Help affect a paradigm shift from existing system to new system

Potential Pitfalls (drawbacks) of Prototyping

1. Get cooperation, buy-in, and understanding

- Some managers and developers may **resist prototyping** if they see it as unnecessary.
- If the purpose of the prototype is not clearly explained, stakeholders may **misinterpret its role** in the process.
- **Solution:** Ensure all involved parties **understand the benefits and iterative nature** of prototyping.

2. Be honest about limitations and do not overpromise

- A highly polished prototype can **mislead stakeholders** into thinking the system is closer to completion than it actually is.
- Users may assume all prototype features will be in the final product, leading to **unrealistic expectations**.
- **Solution:** Always communicate **what the prototype represents and what is subject to change**.

3. Do not overwork your prototype

- Spending **too much time refining a prototype** can slow down progress.
- The prototype should serve as a **testing and feedback tool**, not a **fully developed product**.
- **Solution:** Use a “**good enough**” approach—once the prototype serves its purpose, move forward.

4. Do not “fall in love” with your prototype

- Teams may become **too attached to a prototype**, resisting necessary changes.
- Managers may **want to keep showcasing** a prototype long after its usefulness has ended.
- **Solution:** Set **clear goals and deadlines** for each prototype iteration to prevent excessive attachment.

PROTOTYPES IN TRANSITION TO THE PRODUCT

Graduation Day in the Trenches: The Sacred Passing of the Prototype

- This part describes the moment when the project moves from **prototyping** to actual **software development**. The UX team's work of refining interaction design comes to a close, and the **software engineering (SE) team takes over**.
- Once the UX process ends, the project **shifts focus to final implementation**.
- **Some formative evaluation can still be done**, but now it is with the **real software system** instead of the prototype.
- **Further changes become more expensive and time-consuming**, making it crucial that the prototype was refined properly.
- The UX team may **either disband or move on to a new project** while the development team finalizes the product.

What Happens to the Prototype Code?

The transition from a prototype to a final product raises an important question:
Should the prototype code be reused or discarded?

You Cannot Just Keep the Prototype

- While a prototype **may look functional**, it is **not built for real-world use**.
- Prototypes are created quickly, often **without considering performance, security, or maintainability**.
- The prototype **may have been built using tools or platforms unsuitable** for the final product.

How Do You Reuse the Interaction Design of the Prototype?

- The final product should **faithfully follow the prototype's interaction design** to maintain the user experience.
- The **layout, user flows, and interface elements should remain intact**, even if the underlying code is rewritten.
- Small changes in **font size, button placement, or menu structure** can **negatively impact usability**.

Using a Prototype as a Design Specification

- Many teams use **high-fidelity prototypes** as **interaction design specifications** for developers.
- A **project style guide** (covering colors, fonts, branding, and UI rules) helps ensure **design consistency**.
- Developers must carefully translate the prototype into production software **without losing essential design details**.

The Need for UX and SE Collaboration and Respect

The transition from prototype to product requires **close collaboration between UX designers and software engineers.**

Key challenges in the transition:

- 1. UX teams "own" the interaction design, but not the implementation code.**
- 2. SE teams may modify the design to fit technical constraints, sometimes altering the user experience.**
- 3. UX designers must actively ensure that SE teams follow the intended design, since SE teams may not be experts in user experience.**
- 4. Mutual respect and collaboration are essential to ensure the final product stays true to the prototype's usability goals.**

Do Not Think the UX Team Is Now Done

- Just because the SE team takes over does **not** mean the UX team's job is finished.
- There are **no universal software development standards** that ensure **interaction design is implemented correctly**.
- Developers may **unintentionally change interaction details**, affecting usability.
- UX designers should **stay involved** during implementation to **review and test the final product** for consistency with the prototype.

SOFTWARE TOOLS FOR PROTOTYPING

The Need for Better Software Prototyping Tools

- Ideal prototyping tools should combine speed, flexibility, and ease of **modification** while accurately simulating modern interaction paradigms.
- Despite advances in software, **no single tool fully replaces paper prototyping** in terms of **ease of iteration and spontaneity**.
- Researchers and developers have tried to create **digital alternatives** that maintain the benefits of low-fidelity prototypes while offering interactivity.

Early Efforts in Software Prototyping Tools

User Interface Management Systems (UIMS) in the 1990s

- Early software tools were **complex and programming-heavy**, making them difficult for UX designers to use.
- Examples included **IBM's CUA**, **OpenLook (Next)**, **Toolbook (Asymetrix)**, **Delphi (Borland)**, **Visual Basic (Microsoft)**, and **Dreamweaver (Macromedia)**.
- Some tools were **custom-built by organizations**, like NASA's **TAE Plus**.

Challenges with Early Prototyping Tools

- Required **programming knowledge**, making them **less accessible to UX designers**.
- **Slow iteration cycles** due to coding and debugging.
- **Poor adaptability to changes**, making them **less effective for early-stage design exploration**.

Modern Software Prototyping Tools

Today, there are many tools designed to make prototyping **faster and more intuitive**, such as:

1. **Wireframing Tools** (for low to medium fidelity)
 - Examples: **Balsamiq, OmniGraffle, Microsoft Visio**
 - Used for **quick sketches and layout design** before adding interaction.
2. **Interactive Prototyping Tools** (for medium to high fidelity)
 - Examples: **Axure RP, Figma, Adobe XD, Sketch, InVision**
 - Allow designers to create **clickable prototypes** that simulate real interactions.
3. **Code-Based Prototyping Tools** (for high fidelity and functional prototypes)
 - Examples: **Framer, HTML/CSS/JavaScript, React, Swift, Flutter**
 - Used for **prototyping with real code**, making them closer to final implementation.

The Persistent Challenge in Prototyping

Despite advances, no single tool perfectly balances:

1. Ease of creation and modification (like paper prototypes).
2. Realistic interaction and behavior (like coded prototypes).
3. Rapid iteration without requiring extensive technical knowledge.

This challenge remains an ongoing problem in UX design and HCI (Human-Computer Interaction) research.